

```

1 // ME350 Zombie Defense Sketch - Version 05.1 by Spencer Dickhudt
2 // updated 12-5-2024
3 //
4 //
5
6 ////////////////////////////////////////////////////
7 // DEFINE CONSTANTS AND GLOBAL VARIABLES: //
8 ////////////////////////////////////////////////////
9
10 /** State Machine: **//
11 // CONSTANTS:
12 // Definition of states in the state machine
13 const int CALIBRATE = 1;
14 const int CHOOSE_ACTIVE_TARGET = 2;
15 const int MOVE_TO_TARGET = 3;
16
17 // VARIABLES:
18 // Global variable that keeps track of the state:
19 // Start the state machine in calibration state:
20 int state = CALIBRATE;
21
22 /** Proximity Sensors or Potentiometer: **//
23 // CONSTANTS:
24 // Definition of proximity sensor Limits for each target:
25 const int PROXIMITYSENSE1_MAX = 0; // [proximity sensor counts] Value of prox sensor
26 // 1 when zombie is closest to photosensor
27 const int PROXIMITYSENSE1_MIN = 0; // [proximity sensor counts] Value of prox sensor
28 // 1 when zombie is closest to prox sensor
29 const int PROXIMITYSENSE2_MAX = 0; // [proximity sensor counts] Value of prox sensor
30 // 2 when zombie is closest to photosensor
31 const int PROXIMITYSENSE2_MIN = 0; // [proximity sensor counts] Value of prox sensor
32 // 2 when zombie is closest to prox sensor
33 const int PROXIMITYSENSE3_MAX = 0; // [proximity sensor counts] Value of prox sensor
34 // 3 when zombie is closest to photosensor
35 const int PROXIMITYSENSE3_MIN = 0; // [proximity sensor counts] Value of prox sensor
36 // 3 when zombie is closest to prox sensor
37 const int PROXIMITYSENSE4_MAX = 0; // [proximity sensor counts] Value of prox sensor
38 // 4 when zombie is closest to photosensor
39 const int PROXIMITYSENSE4_MIN = 0; // [proximity sensor counts] Value of prox sensor
40 // 4 when zombie is closest to prox sensor
41
42 const int ProxRange[4][2] = {{PROXIMITYSENSE1_MAX, PROXIMITYSENSE1_MIN},
43                               {PROXIMITYSENSE2_MAX, PROXIMITYSENSE2_MIN},
44                               {PROXIMITYSENSE3_MAX, PROXIMITYSENSE3_MIN},
45                               {PROXIMITYSENSE4_MAX, PROXIMITYSENSE4_MIN}}; // Array
46 // holding prox sensor bounds to improve code readability
47
48 // Definition of target variables for array indexing
49 const int TARGET1 = 0; // For indexing into activeTargets array, activeTargets[0]
50 // corresponds to Target 1
51 const int TARGET2 = 1; // For indexing into activeTargets array, activeTargets[1]
52 // corresponds to Target 2
53 const int TARGET3 = 2; // For indexing into activeTargets array, activeTargets[2]
54 // corresponds to Target 3
55 const int TARGET4 = 3; // For indexing into activeTargets array, activeTargets[3]
56 // corresponds to Target 4
57
58 const int targetArr[4] = { TARGET1, TARGET2, TARGET3, TARGET4 }; // List of target
59 // indices to reduce redundant code
60
61 // Direction variables to represent the movement of each zombie
62 const int FORWARD = 1; // representing the zombie moving towards the photosensor
63 const int BACKWARD = -1; // representing the zombie moving away from the photosensor
64 const int STOPPED = 0; // representing the zombie stalled
65
66 // VARIABLES:

```

```

53 // Variables for updating sensor readings
54 const float alpha = 0.925;           // filter parameter used to weight current and
past readings
55 int stopTimeout = 250;               // [millis] time needed to declare a zombie stopped
56 int noiseLimit = 8;                 // average spread of proximity readings for a
stopped zombie
57 const int lowerNoiseLimit = 5;       // noise limit for near the photosensors
58 const int upperNoiseLimit = 8;       // noise limit for near the prox sensors
59 const int noiseThreshold = 225;      // prox sensor reading at which noise limit
switches from upper to lower

60
61 // A data structure to hold proximity sensor information:
62 struct Sensor{
63     float currVal;                   // the most recent proximity sensor reading
64     float prevVal;                   // the last update of proximity sensor reading
65     unsigned long prevChangeTime;    // [millis] the time at which prevVal last changed
66     int pin;                         // the input pin of the desired prox sensor
67     int direction;                   // the direction at which the zombie on the rail is
traveling
68     int forwardCount;                // number of readings which indicate the zombie is
moving forward
69     int backwardCount;                // number of readings which indicate the zombie is
moving backward
70 };
71
72 Sensor ProxSensors[4];              // An array which holds Sensor objects, representing
each of the prox sensors
73 int potValue = 0;                   // Used to hold the reading of the potentiometer
74
75 int activeTargetIndex = -1;          // index (in the activeTargets array) of the closest
target to the plants
76 int activeTargetPosition = -1;      // [encoder counts] Encoder count of the active target
the linkage is moving toward
77
78 int minIndex = -1;                  // tracking index of the zombie who is closest to the
plants
79 float closestZombieDist = 2;        // [percentage] records the lowest percentage of rail
remaining for any of the zombies
80 int idx = -1;                       // index of target, based on targetArr
81
82 bool WAIT_POS = true;               // tracking if the linkage is moving to the wait position
83
84 unsigned long arrivalTime;           // timer for tracking a wait period upon reaching
a desired position
85 const int targetActivateTime = 350; // time to activate a target
86 float Zombies[4];                   // An array to hold information on locations of
all zombies in play
87
88 // float sampleTime = 1.5           // sample time in ms
89
90 /** Computation of position and velocity: **/
91 // CONSTANTS:
92 // Settings for velocity computation:
93 const int MIN_VEL_COMP_COUNT = 2;    // [encoder counts] Minimal change in motor
position that must happen between two velocity measurements
94 const long MIN_VEL_COMP_TIME = 10000; // [microseconds] Minimal time that must pass
between two velocity measurements
95 // VARIABLES:
96 volatile int motorPosition = 0; // [encoder counts] Current motor position (Declared
'volatile', since it is updated in a function called by interrupts)
97 volatile int encoderStatus = 0; // [binary] Past and Current A&B values of the encoder
(Declared 'volatile', since it is updated in a function called by interrupts)
98 // The rightmost two bits of encoderStatus will store the encoder values from the
current iteration (A and B).
99 // The two bits to the left of those will store the encoder values from the previous
iteration (A_old and B_old).

```

```

100 float motorVelocity      = 0; // [encoder counts / seconds] Current motor velocity
101 int previousMotorPosition = 0; // [encoder counts] Motor position the last time a
    velocity was computed
102 long previousVelCompTime = 0; // [microseconds] System clock value the last time a
    velocity was computed
103
104 /** High-level behavior of the controller: */
105 // CONSTANTS:
106 // Target positions:
107 const int CALIBRATION_VOLTAGE = 0; // [Volt] Motor voltage used during the calibration
    process
108 const int TARGET_1_POSITION   = 0; // [encoder counts] Motor position corresponding to
    first target
109 const int TARGET_2_POSITION   = 0; // [encoder counts] Motor position corresponding to
    second target
110 const int TARGET_3_POSITION   = 0; // [encoder counts] Motor position corresponding to
    third target
111 const int TARGET_4_POSITION   = 0; // [encoder counts] Motor position corresponding to
    fourth target
112 const int WAIT_POSITION       = TARGET_3_POSITION; // [encoder counts] Motor position
    corresponding to a wait position (when no targets are active)
113 const int LOWER_BOUND         = TARGET_1_POSITION; // [encoder counts] Position of the
    left end stop
114 const int UPPER_BOUND         = TARGET_4_POSITION; // [encoder counts] Position of the
    right end stop
115 const int TARGET_BAND         = 5; // [encoder counts] "Close enough" range when moving
    towards a target.
116
117 // List of target positions to reduce redundant code
118 const int targetPos[4] = {TARGET_1_POSITION, TARGET_2_POSITION, TARGET_3_POSITION,
    TARGET_4_POSITION};
119
120 // Timing:
121 //const long WAIT_TIME        = 0; // [microseconds] Time waiting for the target to
    drop.
122
    // TBD - implement a timer so if the target
    // doesn't drop, the linkage moves to a different
    // target
123 // VARIABLES:
124 //unsigned long startWaitTime; // [microseconds] System clock value
125
126 /** PID Controller */
127 // CONSTANTS:
128 const float KP                = 0; // [Volt / encoder counts] P-Gain
129 const float KI                = 0; // [Volt / (encoder counts *
    seconds)] I-Gain
130 const float KD                = 0; // [Volt * seconds / encoder
    counts] D-Gain
131 const float SUPPLY_VOLTAGE    = 0; // [Volt] Supply voltage at the
    HBridge
132 const float FRICTION_COMP_VOLTAGE = 0; // [Volt] Voltage needed to
    overcome friction
133 // VARIABLES:
134 int desiredPosition = 0; // [encoder counts] desired motor position
135 float positionError = 0; // [encoder counts] Position error
136 float integralError = 0; // [encoder counts * seconds] Integrated position error
137 float velocityError = 0; // [encoder counts / seconds] Velocity error
138 float desiredVoltage = 0; // [Volt] Desired motor voltage
139 int motorCommand = 0; // [0-255] PWM signal sent to the motor
140 unsigned long executionDuration = 0; // [microseconds] Time between this and the
    previous loop execution. Variable used for integrals and derivatives
141 unsigned long lastExecutionTime = 0; // [microseconds] System clock value at the moment
    the loop was started the last time
142
143 /** Pin assignment: */
144 // CONSTANTS:

```

```

145 const int PIN_NR_ENCODER_A      = 2;  // Never change these, since the interrupts are
    attached to pins 2 and 3
146 const int PIN_NR_ENCODER_B      = 3;  // Never change these, since the interrupts are
    attached to pins 2 and 3
147 const int PIN_NR_ON_OFF_SWITCH  = 5;  // Connected to toggle switch (turns mechanism
    on and off)
148 const int PIN_NRL_LIMIT_SWITCH  = 8;  // Connected to limit switch (mechanism
    calibration)
149 const int PIN_NR_PWM_OUTPUT      = 11; // Connected to H Bridge (controls motor speed)
150 const int PIN_NR_PWM_DIRECTION_1 = 12; // Connected to H Bridge (controls motor
    direction)
151 const int PIN_NR_PWM_DIRECTION_2 = 13; // Connected to H Bridge (controls motor
    direction)
152 const int PIN_PROXSENSE1        = A0; // Connected to proximity sensor 1
153 const int PIN_PROXSENSE2        = A1; // Connected to proximity sensor 2
154 const int PIN_PROXSENSE3        = A2; // Connected to proximity sensor 3
155 const int PIN_PROXSENSE4        = A3; // Connected to proximity sensor 4
156 const int PIN_POTENTIOMETER     = A4; // Connected to potentiometer used to test
    target positions
157
158
159 // Add this line of code if you want to use two limit switches; const int
    PIN_NRL_LIMIT_SWITCH_2 = 11
160 // ^KEEP IN MIND THAT YOU HAVE TO ADD CODE DOWNSTREAM (FOR EXAMPLE YOU NEED TO ADD THIS
    VARIABLE IN THE DECLARATION SECTION
161
162 // End of CONSTANTS AND GLOBAL VARIABLES
163
164
165 ///////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
    /
166 // The setup() function is called when a sketch starts. Use it to initialize variables,
    //
167 // pin modes, start using libraries, etc. The setup function will only run once, after
    //
168 // each powerup or reset of the Arduino board:
    //
169 ///////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
    /
170 void setup() {
171     // Declare which digital pins are inputs and which are outputs:
172     pinMode(PIN_NR_ENCODER_A,      INPUT_PULLUP);
173     pinMode(PIN_NR_ENCODER_B,      INPUT_PULLUP);
174     pinMode(PIN_NR_ON_OFF_SWITCH,   INPUT);
175     pinMode(PIN_NRL_LIMIT_SWITCH,   INPUT);
176     pinMode(PIN_PROXSENSE1,        INPUT);
177     pinMode(PIN_PROXSENSE2,        INPUT);
178     pinMode(PIN_PROXSENSE3,        INPUT);
179     pinMode(PIN_PROXSENSE4,        INPUT);
180     pinMode(PIN_NR_PWM_OUTPUT,      OUTPUT);
181     pinMode(PIN_NR_PWM_DIRECTION_1, OUTPUT);
182     pinMode(PIN_NR_PWM_DIRECTION_2, OUTPUT);
183     pinMode(PIN_POTENTIOMETER,      INPUT);
184
185     // Turn on the pullup resistors on the encoder channels
186     digitalWrite(PIN_NR_ENCODER_A, HIGH);
187     digitalWrite(PIN_NR_ENCODER_B, HIGH);
188
189     // Activate interrupt for encoder pins.
190     // If either of the two pins changes, the function 'updateMotorPosition' is called:
191     attachInterrupt(0, updateMotorPosition, CHANGE); // Interrupt 0 is always attached to
    digital pin 2
192     attachInterrupt(1, updateMotorPosition, CHANGE); // Interrupt 1 is always attached to
    digital pin 3
193
194     // Begin serial communication for monitoring.

```

```

195 Serial.begin(115200);
196 Serial.println("Start Executing Program.");
197
198 // Initialize sensor values
199 ProxSensors[TARGET1].prevVal = analogRead(PIN_PROXSENSE1);
200 ProxSensors[TARGET2].prevVal = analogRead(PIN_PROXSENSE2);
201 ProxSensors[TARGET3].prevVal = analogRead(PIN_PROXSENSE3);
202 ProxSensors[TARGET4].prevVal = analogRead(PIN_PROXSENSE4);
203
204 // Initialize prox sensor pins
205 ProxSensors[TARGET1].pin = PIN_PROXSENSE1;
206 ProxSensors[TARGET2].pin = PIN_PROXSENSE2;
207 ProxSensors[TARGET3].pin = PIN_PROXSENSE3;
208 ProxSensors[TARGET4].pin = PIN_PROXSENSE4;
209
210 // Set initial output to the motor to 0
211 analogWrite(PIN_NR_PWM_OUTPUT, 0);
212 }
213 // End of function setup()
214
215
216 ////////////////////////////////////////
217 // After going through the setup() function, which initializes and sets the initial
values, //
218 // the loop() function does precisely what its name suggests, and loops
consecutively, //
219 // allowing your program to sense and respond. Use it to actively control the Arduino
board. //
220 ////////////////////////////////////////
221 void loop() {
222     // Determine the duration it took to execute the last loop. This time is used
for integration and for monitoring the loop time via the serial monitor.
223     executionDuration = micros() - lastExecutionTime;
224     lastExecutionTime = micros();
225
226     // Speed Computation:
227     if ((abs(motorPosition - previousMotorPosition) > MIN_VEL_COMP_COUNT) || (micros() -
previousVelCompTime) > MIN_VEL_COMP_TIME){
228         // If at least a minimum time interval has elapsed or
229         // the motor has travelled through at least a minimum angle ...
230         // .. compute a new value for speed:
231         // (speed = delta angle [encoder counts] divided by delta time [seconds])
232         motorVelocity = (double)(motorPosition - previousMotorPosition) * 1000000 /
233             (micros() - previousVelCompTime);
234         // Remember this encoder count and time for the next iteration:
235         previousMotorPosition = motorPosition;
236         previousVelCompTime = micros();
237     }
238     // *****//
239     // We first update the sensors so our direction information stays accurate
240
241     // Loop through each of the sensors
242     for (int i = 0; i < sizeof(targetArr)/sizeof(int); i++) {
243
244         idx = targetArr[i];
245         // Updates the current sensor reading using a low pass filter
246         ProxSensors[idx].currVal = alpha * ProxSensors[idx].currVal + (1 - alpha) *
analogRead(ProxSensors[idx].pin);
247
248         // Adjusts the noise Limit based on distance to sensor (since values are not linear)
249         if (ProxSensors[idx].currVal >= noiseThreshold) {
250             noiseLimit = upperNoiseLimit;
251         } else {
252             noiseLimit = lowerNoiseLimit;
253         }

```

```

254     }
255
256     // Check if sensor is stopped
257     if (abs(ProxSensors[idx].currVal - ProxSensors[idx].prevVal) < noiseLimit) {
258         // if not enough time has passed to declare stopped or already stopped continue
        loop
259         if (millis() - ProxSensors[idx].prevChangeTime < stopTimeout ||
            ProxSensors[idx].direction == STOPPED) {
260             ProxSensors[i].forwardCount = 0;
261             ProxSensors[i].backwardCount = 0;
262             continue;
263         } else {
264             // the prox sensor has been stopped for long enough that we know it is stopped
265             ProxSensors[idx].direction = STOPPED;
266         }
267     } else if (ProxSensors[idx].currVal - ProxSensors[idx].prevVal < 0){
268         // Target is moving forward, as readings decrease moving further away from the
        sensor
269         ProxSensors[i].forwardCount += 1;
270         // Target is moving forward
271         if (ProxSensors[i].forwardCount > 3){
272             ProxSensors[i].direction = FORWARD;
273
274             ProxSensors[i].prevVal = ProxSensors[i].currVal;
275             ProxSensors[i].prevChangeTime = millis();
276         }
277     } else {
278         // Target must be moving backward
279         ProxSensors[i].backwardCount += 1;
280         // Target is moving forward
281         if (ProxSensors[i].backwardCount > 3){
282             ProxSensors[i].direction = BACKWARD;
283
284             ProxSensors[i].prevVal = ProxSensors[i].currVal;
285             ProxSensors[i].prevChangeTime = millis();
286         }
287     }
288 }
289 // *****//
290
291 //*****//
292 // The state machine:
293 switch (state) {
294     //*****//
295     // In the CALIBRATE state, we move the mechanism to a position outside of the
296     // work space (towards the limit switch). Once the limit switch is on and
297     // the motor has stopped turning, we know that we are against the end stop
298     case CALIBRATE:
299         // We don't have to do anything here since this state is only used to set
300         // a fixed output voltage. This happens further below.
301
302         // Decide what to do next:
303         if (digitalRead(PIN_NRL_LIMIT_SWITCH)==HIGH && motorVelocity==0) {
304             // We reached the endstop. Update the motor position to the limit:
305             // (NOTE: If the limit switch is on the right, this must be UPPER_BOUND)
306             motorPosition = LOWER_BOUND;
307             // Reset the error integrator:
308             integralError = 0;
309             // Calibration is finalized. Transition into DETERMINE_ACTIVE_TARGETS state
310             Serial.println("State transition from CALIBRATE to CHOOSE_ACTIVE_TARGET");
311             state = CHOOSE_ACTIVE_TARGET;
312         }
313
314         // Otherwise we continue calibrating
315         break;
316

```

```

317 //*****//
318 // In the DETERMINE_ACTIVE_TARGETS state, we use the most recent reading of the
    photosensors to
319 // calculate the location of each zombie.
320 // We then select an active target to move toward based on which zombie is closest
    to the plants.
321 // We default to WAIT_POSITION if no targets are active.
322 case CHOOSE_ACTIVE_TARGET:
323
324     // Decide what zombie is closest to the photosensor
325     // Current closest zombie is out of bounds
326     minIndex = -1;
327     // Set distance to 2 to be above any possible percentage values + noise
328     closestZombieDist = 2;
329
330     for (int i = 0; i < sizeof(targetArr)/sizeof(int); i++) {
331         // Use the target array for the proper indexes into other arrays
332         idx = targetArr[i];
333
334         // Calculate the distance to the front of the rail by taking a percentage of how
            much rail
335         // the zombie has left to travel.
336         Zombies[idx] = (ProxSensors[idx].currVal - ProxRange[idx][1]) /
            (ProxRange[idx][0] - ProxRange[idx][1]);
337
338         // Check to see if the zombie is traveling forward and it is closer than the
            previous zombie
339         if (ProxSensors[idx].direction == FORWARD && Zombies[idx] < closestZombieDist) {
340             // update the newest minimum value
341             closestZombieDist = Zombies[idx];
342             // update the index of the active target (in the targetArr array)
343             minIndex = i;
344         }
345     }
346
347     // if the min index points to a valid target, move to that target
348     if (minIndex >= 0) {
349         activeTargetIndex = targetArr[minIndex];
350         activeTargetPosition = targetPos[activeTargetIndex];
351         WAIT_POS = false;
352         // Serial.println("Setting position to something other than wait");
353     } else {
354         activeTargetPosition = WAIT_POSITION;
355         WAIT_POS = true;
356         // Serial.println("Setting position to wait position");
357     }
358
359     state = MOVE_TO_TARGET;
360     // Serial.println("Switching state to MOVE_TO_TARGET");
361
362     // Otherwise, we stay in DETERMINE_ACTIVE_TARGETS
363     break;
364
365 //*****//
366 // In the MOVE_TO_TARGET state, we select an active target and move toward it, or
    // move toward Target 3 (a default position) if there is no active target
367 case MOVE_TO_TARGET:
368     // Serial.println("Inside MOVE_TO_TARGET");
369     desiredPosition = activeTargetPosition;
370
371     if (motorPosition <= activeTargetPosition + TARGET_BAND && motorPosition >=
        activeTargetPosition - TARGET_BAND) {
372
373         if (millis() - arrivalTime > targetActivateTime || WAIT_POS){
374             state = CHOOSE_ACTIVE_TARGET;

```

```

377     }
378     } else {
379         arrivalTime = millis();
380     }
381
382     break;
383
384     //*****//
385     //*****//
386     // We should never reach the next bit of code, which would mean that the state
387     // we are currently in doesn't exist. So if it happens, throw an error and
388     // stop the program:
389     default:
390         Serial.println("Statemachine reached at state that it cannot handle. ABORT!!!!");
391         Serial.print("Found the following unknown state: ");
392         Serial.println(state);
393         while (1); // infinite loop to halt the program
394     break;
395 }
396 // End of the state machine.
397 //*****//
398
399 //*****//
400 // Recalibrate if we are in the leftmost position
401 if (digitalRead(PIN_NRL_LIMIT_SWITCH)==HIGH && motorVelocity==0) {
402     // We reached the endstop. Update the motor position to the limit:
403     // (NOTE: If the limit switch is on the right, this must be UPPER_BOUND)
404     motorPosition = LOWER_BOUND;
405     // Reset the error integrator:
406     integralError = 0;
407     Serial.println("Limit Switch hit");
408 }
409
410
411 //*****//
412 // Position Controller
413 if (digitalRead(PIN_NR_ON_OFF_SWITCH)==HIGH) {
414     // If the toggle switch is on, run the controller:
415
416     /** PID control: **/
417     // Compute the position error [encoder counts]
418     positionError = desiredPosition - motorPosition;
419     // Compute the integral of the position error [encoder counts * seconds]
420     integralError = integralError + positionError * (float)(executionDuration) /
421     1000000;
422     // Compute the velocity error (desired velocity is 0) [encoder counts / seconds]
423     velocityError = 0 - motorVelocity;
424     // This is the actual controller function that uses the error in
425     // position and velocity and the integrated error and computes a
426     // desired voltage that should be sent to the motor:
427     desiredVoltage = KP * positionError +
428                     KI * integralError +
429                     KD * velocityError;
430
431     /** Feedforward terms: **/
432     // Compensate for friction. That is, if we know the direction of
433     // desired motion, add a base command that helps with moving in this
434     // direction:
435     if (positionError < -5) {
436         desiredVoltage = desiredVoltage - FRICTION_COMP_VOLTAGE;
437     }
438     if (positionError > +5) {
439         desiredVoltage = desiredVoltage + FRICTION_COMP_VOLTAGE;
440     }
441     // Anti-Wind-Up

```



```

442     if (abs(desiredVoltage)>SUPPLY_VOLTAGE) {
443         // If we are already saturating our output voltage, it does not make
444         // sense to keep integrating the error (and thus ask for even higher
445         // and higher output voltages). Instead, stop the integrator if the
446         // output saturates. We do this by reversing the summation at the
447         // beginning of this function block:
448         integralError = integralError - positionError * (float)(executionDuration) /
1000000;
449     }
450     // End of 'if(onOffSwitch==HIGH)'
451
452     // Override the computed voltage during calibration. In this state, we simply apply
a
453     // fixed voltage to move against one of the end-stops.
454     if (state==CALIBRATE) {
455         // add calibration code here
456
457     }
458 } else {
459     // Otherwise, the toggle switch is off, so do not run the controller,
460     // stop the motor...
461     desiredVoltage = 0;
462     // .. and reset the integrator of the error:
463     integralError = 0;
464     // Produce some debugging output:
465     Serial.println("The toggle switch is off. Motor Stopped.");
466 }
467 // End of else onOffSwitch==HIGH
468
469 /** Send signal to motor */
470 // Convert from voltage to PWM cycle:
471 motorCommand = int(abs(desiredVoltage * 255 / SUPPLY_VOLTAGE));
472 // Clip values larger than 255
473 if (motorCommand > 255) {
474     motorCommand = 255;
475 }
476 // Send motor signals out
477 analogWrite(PIN_NR_PWM_OUTPUT, motorCommand);
478 // Determine rotation direction
479 if (desiredVoltage >= 0) {
480     // If voltage is positive ...
481     // ... turn forward
482     digitalWrite(PIN_NR_PWM_DIRECTION_1,LOW); // rotate forward
483     digitalWrite(PIN_NR_PWM_DIRECTION_2,HIGH); // rotate forward
484 } else {
485     // ... otherwise turn backward:
486     digitalWrite(PIN_NR_PWM_DIRECTION_1,HIGH); // rotate backward
487     digitalWrite(PIN_NR_PWM_DIRECTION_2,LOW); // rotate backward
488 }
489 // End of Position Controller
490 //*****//
491
492 // Print out current controller state to Serial Monitor.
493 printStateToSerial();
494 }
495 // End of main loop
496 //*****//
497
498
499 ///////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
500 // This is a function to update the encoder count in the Arduino. //
501 // It is called via an interrupt whenever the value on encoder //
502 // channel A or B changes. //
503 ///////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
504 void updateMotorPosition() {
505     // Bitwise shift left by one bit, to make room for a bit of new data:

```

```

506     encoderStatus <= 1;
507     // Use a compound bitwise OR operator (|=) to read the A channel of the encoder (pin 2)
508     // and put that value into the rightmost bit of encoderStatus:
509     encoderStatus |= digitalRead(2);
510     // Bitwise shift left by one bit, to make room for a bit of new data:
511     encoderStatus <= 1;
512     // Use a compound bitwise OR operator (|=) to read the B channel of the encoder (pin
513     // and put that value into the rightmost bit of encoderStatus:
514     encoderStatus |= digitalRead(3);
515     // encoderStatus is truncated to only contain the rightmost 4 bits by using a
516     // bitwise AND operator on mstatus and 15(=1111):
517     encoderStatus &= 15;
518     if (encoderStatus==2 || encoderStatus==4 || encoderStatus==11 || encoderStatus==13) {
519         // the encoder status matches a bit pattern that requires counting up by one
520         motorPosition++;           // increase the encoder count by one
521     }
522     else if (encoderStatus == 1 || encoderStatus == 7 || encoderStatus == 8 ||
523     encoderStatus == 14) {
524         // the encoder status does not match a bit pattern that requires counting up by
525         one.
526         // Since this function is only called if something has changed, we have to count
527         downwards
528         motorPosition--;           // decrease the encoder count by one
529     }
530 }
531 // End of function updateMotorPosition()
532
533 ///////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
534 // This function sends a status of the controller to the serial //
535 // monitor. Each character will take 85 microseconds to send, so //
536 // be selective in what you write out: //
537 ///////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
538 void printStateToSerial() {
539     //*****//
540     // Send a status of the controller to the serial monitor.
541     // Each character will take 85 microseconds to send, so be selective
542     // in what you write out:
543
544     //Serial.print("State Number:  [CALIBRATE = 1; DETERMINE_ACTIVE_TARGETS = 2;
545     MOVE_TO_TARGET = 3]: ");
546     Serial.print("State#: ");
547     Serial.print(state);
548
549     //Serial.print("Power switch [on/off]: ");
550     //Serial.print(" PWR: ");
551     //Serial.print(digitalRead(PIN_NR_ON_OFF_SWITCH));
552
553     //Serial.print("      Motor Position [encoder counts]: ");
554     Serial.print(" MP: ");
555     Serial.print(motorPosition);
556
557     //Serial.print("      Motor Velocity [encoder counts / seconds]: ");
558     Serial.print(" MV: ");
559     Serial.print(motorVelocity);
560
561     //Serial.print("      Encoder Status [4 bit value]: ");
562     //Serial.print(" ES: ");
563     //Serial.print(encoderStatus);
564
565     //Serial.print("      Target Position [encoder counts]: ");
566     Serial.print(" DP: ");
567     Serial.print(desiredPosition);
568
569     // //Serial.print("      Position Error [encoder counts]: ");

```

```

567 // Serial.print(" PE: ");
568 // Serial.print(positionError);
569
570 // //Serial.print(" Integrated Error [encoder counts * seconds]: ");
571 // Serial.print(" IE: ");
572 // Serial.print(integralError);
573
574 // //Serial.print(" Velocity Error [encoder counts / seconds]: ");
575 // Serial.print(" VE: ");
576 // Serial.print(velocityError);
577
578 //Serial.print(" Desired Output Voltage [Volt]: ");
579 Serial.print(" DV: ");
580 Serial.print(desiredVoltage);
581
582 //Serial.print(" Motor Command [0-255]: ");
583 //Serial.print(" MC: ");
584 //Serial.print(motorCommand);
585
586 //Serial.print(" Execution Duration [microseconds]: ");
587 //Serial.print(" ED: ");
588 //Serial.print(executionDuration);
589
590 //Serial.print(" Zombie Location [% of rail left to travel]: ");
591 Serial.print(" ZL: ");
592 for (int i = 0; i < sizeof(Zombies)/sizeof(float); i++) {
593     Serial.print(Zombies[i]);
594     Serial.print(" ");
595 }
596
597 //Serial.print(" Proxsensor Direction[STOPPED or FORWARD or BACKWARD]: ");
598 Serial.print(" DR: ");
599 for (int i = 0; i < sizeof(Zombies)/sizeof(float); i++) {
600     Serial.print(ProxSensors[i].direction);
601     Serial.print(" ");
602 }
603
604 // ALWAYS END WITH A NEWLINE. SERIAL MONITOR WILL CRASH IF NOT
605 Serial.println(); // new line
606 }
607 // End of Serial Out
608
609 ///////////////////////////////////////////////////////////////////
610 // This function returns the average of the integer array //
611 // pointed to by array_ptr and of length len. //
612 ///////////////////////////////////////////////////////////////////
613 float average (int * array_ptr, int len) {
614     long sum = 0L;
615     for (int i = 0; i < len; i++) {
616         sum += array_ptr[i];
617     }
618     return ((float) sum)/len;
619 }
620 // End of average

```